Abstract

In an era of increasing digital surveillance and data interception, secure communication has become a critical necessity. This project presents a video steganography system built in Java that conceals secret text messages within ordinary video files, making hidden data completely invisible to the human eye and undetectable without the decoding program.

The core problem addressed is the need for covert communication — transmitting sensitive information without revealing that any communication is taking place at all. Unlike encryption, which hides the contents of a message, steganography hides the very existence of the message.

The solution uses LSB (Least Significant Bit) substitution to embed message characters into pixel colour channels across randomly selected video frames. A 16-bit key, generated during encoding, is stored directly inside the video itself, enabling fully self-contained decoding with no external key transfer required.

The system encodes a message into a 329-frame 1280×534 video in approximately 9 seconds and decodes it in approximately 1 second. Pixel modification is limited to 0.00044% of pixels per character with a maximum colour deviation of ±1 out of 255, ensuring zero perceptible visual difference between the original and encoded video.

1. Project Overview

Steganography is the practice of hiding secret information within ordinary, non-secret data or files to avoid detection. The word originates from the Greek words steganos (covered) and graphein (writing). Unlike cryptography which scrambles a message, steganography conceals the fact that a message exists at all. Digital steganography has applications in covert communication, digital watermarking, copyright protection, and cybersecurity research.

With the rise of digital communication and surveillance, there is a growing need for methods that allow information to be transmitted without attracting attention. Video files are particularly well-suited for steganography because they contain a massive amount of redundant pixel data across hundreds of frames, providing ample space to hide information with minimal impact on visual quality.

This project demonstrates a practical implementation of video steganography using Java and OpenCV. It serves as both an educational exploration of information hiding techniques and a working proof-of-concept for covert data transmission. The system is significant because it achieves true invisibility — the encoded video is indistinguishable from the original even under direct comparison.

2. Problem Statement

2.1 Business Problem

Organizations and individuals transmitting sensitive information over public networks face the risk of interception, surveillance, and data theft. Conventional encrypted communication, while secure, is visible — an intercepted encrypted message signals that sensitive data is being transmitted, inviting further investigation or legal scrutiny in certain jurisdictions. There is a need for a communication method that conceals not just the content but the existence of the communication itself. The cost of a data breach, reputational damage from

exposed communications, and the legal risks in surveillance-heavy environments make covert communication a real operational need.

2.2 Technical Problem

Existing steganography tools are primarily image-based, limiting the amount of data that can be hidden. Video-based steganography tools are scarce, often require complex dependencies, or use lossy compression that destroys the hidden data. Key management is also a common technical gap — most systems require the key to be shared separately, creating an additional attack surface. The specific technical challenges in this project included OpenCV's Video Writer backend conflicts on Windows that silently failed to write output, codec availability issues where standard codecs like DIB and raw AVI were unavailable, and the need for a lossless codec to preserve LSB-level pixel data.

3. Scope

- What is included in the project

    1. Encoding secret text messages into video files using LSB substitution
    2. Decoding and recovering hidden messages from encoded videos
    3. Self-contained key storage inside the video file itself
    4. Support for any input MP4 video file
    5. Lossless AVI output using FFV1 codec
    6. Cross-machine decoding (encode on one machine, decode on another)
    7. Windows platform support

- What is excluded (limitations/boundaries)

    1. Non-ASCII / Unicode character support
    2. Audio steganography
    3. Image steganography
    4. Encryption of the hidden message (steganography only, not cryptography)
    5. Mobile or web deployment
    6. Real-time video stream steganography
    7. Detection resistance against dedicated steganalysis tools

4. Objectives

- Primary objective

  To build a working video steganography system that can hide and recover secret text messages inside video files with no perceptible visual difference between the original and encoded video.

- Secondary objectives

  1. Implement a secure pseudorandom frame selection mechanism using a seeded key
  2. Store the key inside the video itself to eliminate the need for external key sharing
  3. Ensure pixel-level lossless preservation using FFV1 codec
  4. Demonstrate cross-machine decoding via file transfer

4.1 Measurable Goals

- Encoding time under 10 seconds for a 329-frame 1280×534 video ✓ (achieved ~9 seconds)
- Decoding time under 5 seconds ✓ (achieved ~3 seconds)
- Zero visible difference between original and encoded video ✓
- 100% message recovery accuracy ✓
- Pixel modification rate below 0.001% per character ✓ (achieved 0.00044%)

5 System Architecture & Tools Used

5.1 Encoder Architecture

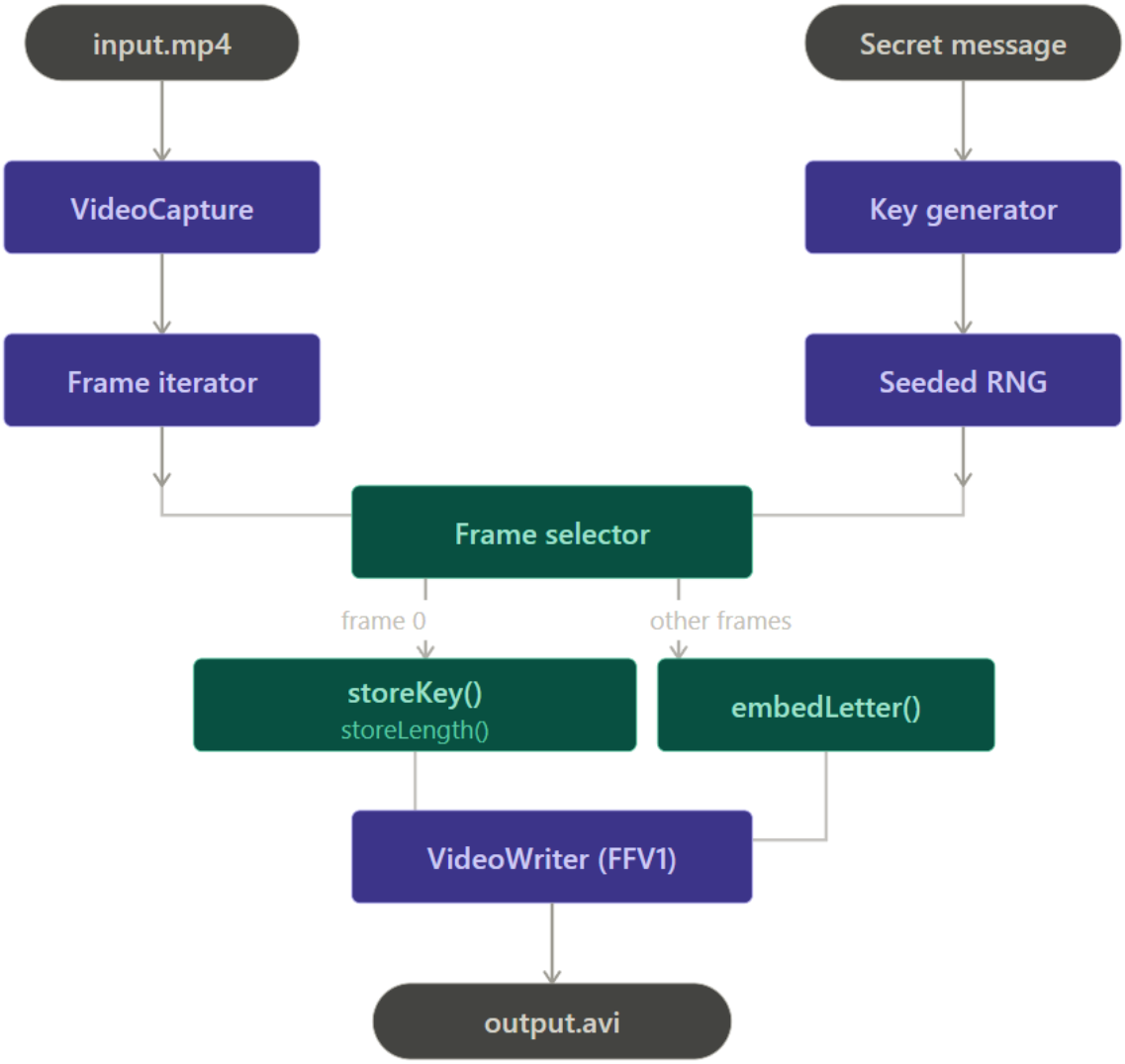

Fig 5.1 Encoder Architecture

5.2 Decoder Architecture

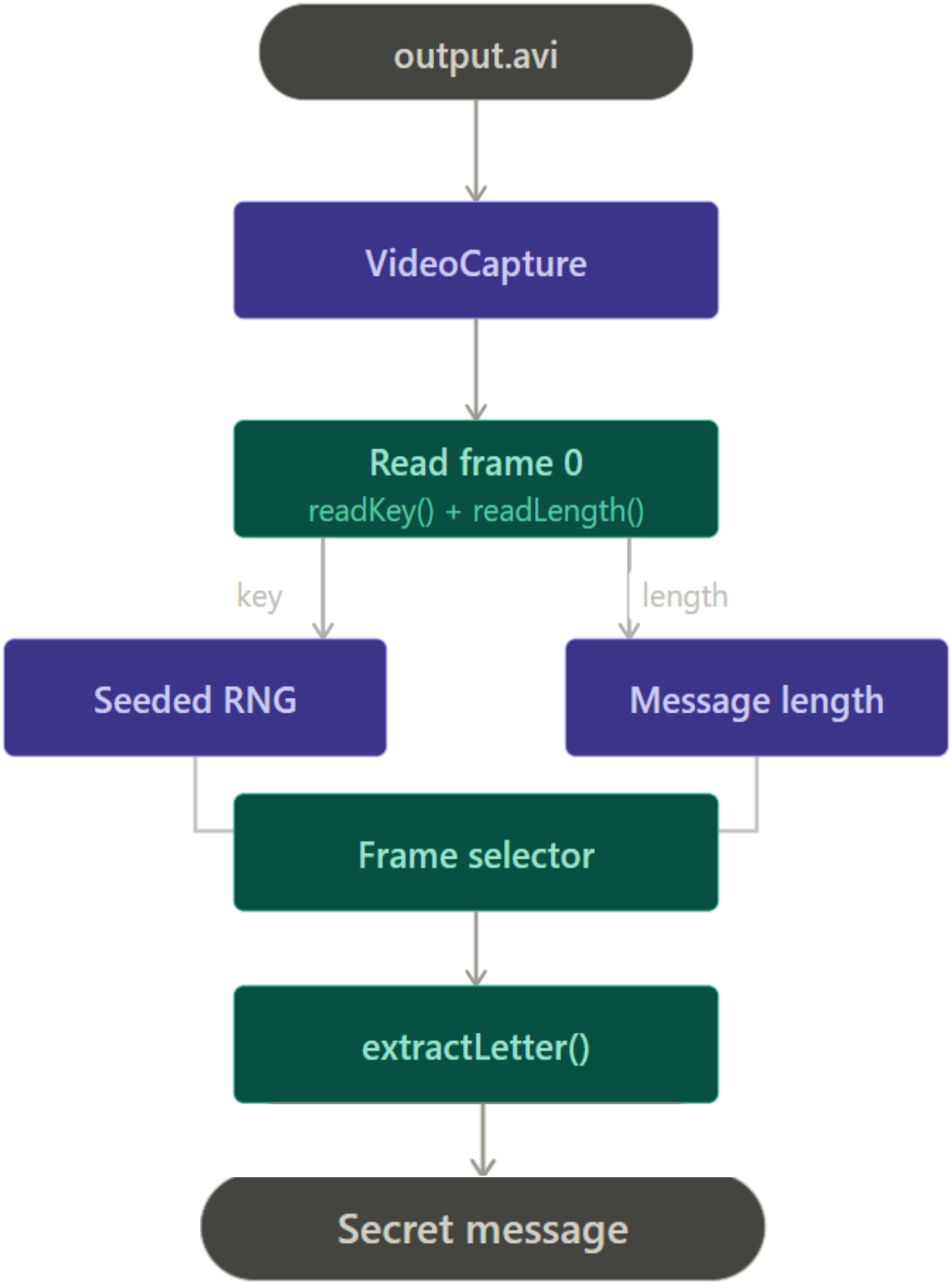

Fig 5.2 Decoder Architecture

5.3 Components explanation (frontend, backend, database, APIs, etc.)

1. VideoCapture — OpenCV component that reads video frames from input.mp4 one by one
2. Key Generator — Java Random class generating a 16-bit integer (0–65535)
3. Seeded RNG — Random instance seeded with the key, producing identical frame sequences on both encoder and decoder
4. Frame Selector — picks one unique frame per character, excluding frame 0 which is reserved for metadata
5. storeKey / storeLength — writes 16-bit values into LSBs of frame 0 pixels at columns 0–5 and 6–11 respectively
6. embedLetter — writes 8 ASCII bits into LSBs of pixels starting at column 12 of the selected frame
7. VideoWriter — OpenCV component writing modified frames to output.avi using FFV1 lossless codec

5.4 Tools & Technologies

Category	**Tools / Technologies**
Programming	Java JDK 11+
Computer Vision	OpenCV 4.12.0
Video Codec	FFV1 Lossless (via FFmpeg)
Libraries	opencv-4120.jar
Native Binaries	opencv_java4120.dll, opencv_videoio_ffmpeg4120_64.dll

7. Explanation of Project Modules

Module 1: Key Generation & Metadata Storage

- Description

  Handles the generation of the random key and the storage of metadata (key and message length) into a random frame in the video.

157	153	174	168	150	152	129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Secret Key Hidden						129	151	172	161	155	156
155	182	163	74	75	62	33	17	110	210	180	154
180	180	50	14	34	6	10	33	48	106	159	181
206	109	5	124	131	111	120	204	166	15	56	180
194	68	137	251	237	239	239	228	227	87	71	201
172	105	207	233	233	214	220	239	228	98	74	206
188	88	179	209	185	215	211	158	139	75	20	169
189	97	165	84	10	168	134	11	31	62	22	148
199	168	191	193	158	227	178	143	182	106	36	190
205	174	155	252	236	231	149	178	228	43	95	234
190	216	116	149	236	187	86	150	79	38	218	241
190	224	147	108	227	210	127	102	36	101	255	224
190	214	173	66	103	143	96	50	2	109	249	215
187	196	235	75	1	81	47	0	6	217	255	211
183	202	237	145	0	0	12	108	200	138	243	236
195	206	123	207	177	121	123	200	175	13	96	218

Fig 7.1 Key Storage

- Functionality

  A 16-bit random key is generated using Java's Random class. This key and the message length are each stored as 16-bit values in the LSBs of frame 0's pixel channels. The key occupies columns 0–5 (18 available bit slots, 16 used) and the length occupies columns 6–11, with no overlap between the two regions.

- Input :

  Video frame 0 (Mat object), integer key, integer message length

- Output :

  Modified frame 0 with key and length embedded in pixel LSBs

Module 2: Pseudorandom Frame Selection

- Description

  Determines which frames of the video will carry each character of the secret message.

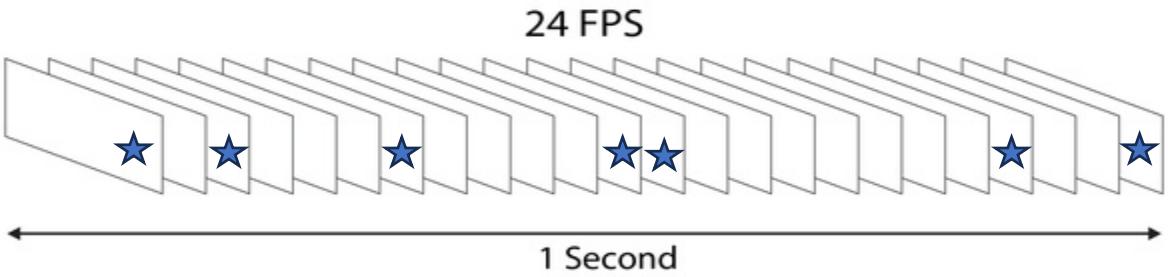

Fig 7.2 Random Frame Selection

  Here the stars indicate the random frames selected to store the secret message

- Functionality

  Using the generated key as a seed, a Random instance produces a sequence of frame indices. Each index is unique (no frame is used twice), and frame 0 is excluded (reserved for metadata). The resulting array of frame indices is sorted so characters are embedded in ascending frame order, matching the decoding sequence.

- Input :

  Key (integer), total frame count, message length

- Output :

  Sorted integer array of frame indices, one per character

Module 3: LSB Embedding (Encoder)

- Description

  Hides each character of the secret message inside the pixels of its assigned frame.

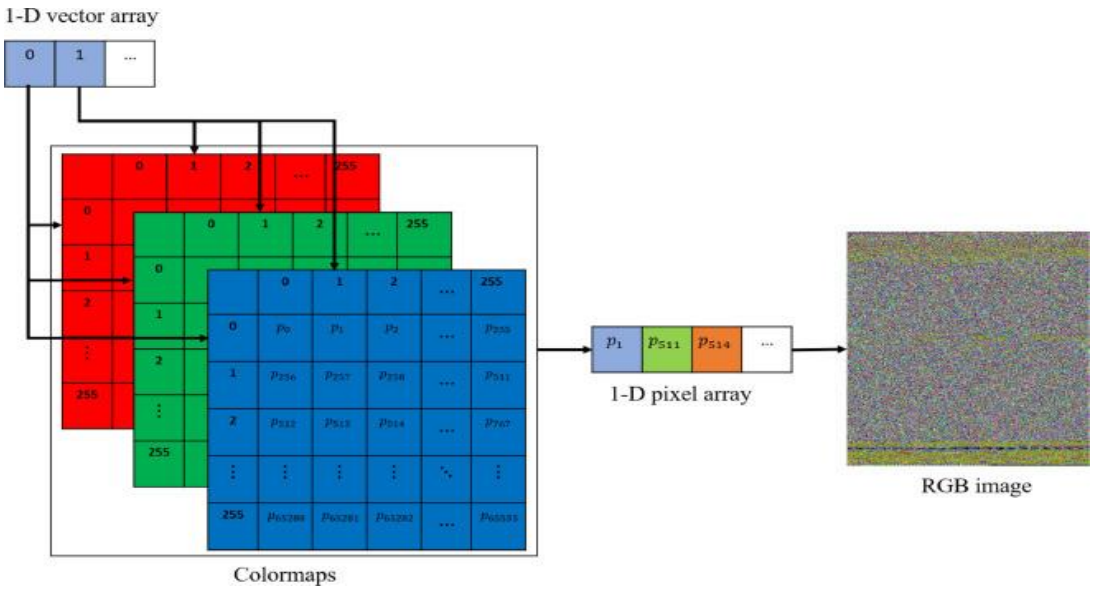

Fig 7.3 RGB Pixel Representation

- Functionality

Each character is converted to its 8-bit ASCII binary representation. Starting at column 12 of the assigned frame (past the metadata zone), the 8 bits are written one by one into the LSB of each colour channel (B, G, R) of consecutive pixels. A colour value like 10110100 becomes 10110101 if the bit being stored is 1 — a change of at most 1 out of 255.

Example :

Character = ‘A’

ASCII of ‘A’ = 65

Binary of 65 = 01000001

Pixel	Channel	Original	LSB to store	New value
1	Blue	1011010**0**	0	1011010**0**
1	Green	1100101**1**	1	1100101**1**
1	Red	0110111**1**	0	0110111**0**
2	Blue	1011010**1**	0	1011010**0**
2	Green	1100101**0**	0	1100101**0**
2	Red	0110111**1**	0	0110111**0**
3	Blue	1011010**0**	0	1011010**0**
3	Green	1100101**1**	1	1100101**1**

Table 7.3 Pixel Encoding

- Input :

  Video frame (Mat object), character to embed

- Output :

  Modified frame with character hidden in pixel LSBs

Module 4: LSB Extraction (Decoder)

- Description

  Recovers each character from its assigned frame by reading back the LSBs.

- Functionality

  The decoder visits each frame identified by the frame selector, reads the LSB of each colour channel starting at column 12, reconstructs the 8-bit binary value, and converts it back to a character. The process is the exact mirror of the embedding step.

- Input :

Fig 7.4a Encoding UI

- Output :

Recovered character

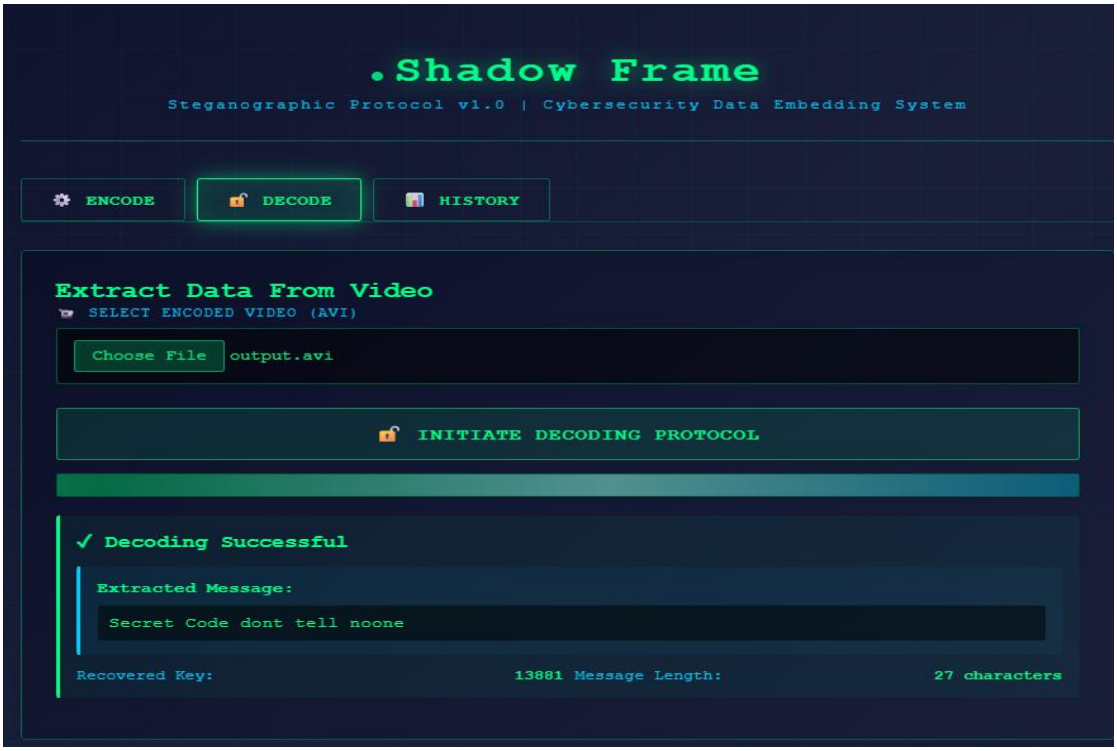

Fig 7.4b Decoding UI

Module 5: Video I/O

- Description

  Handles reading the input video and writing the encoded output video.

- Functionality

  VideoCapture reads input.mp4 frame by frame. VideoWriter writes each modified frame to output.avi using the FFV1 lossless codec via the FFMPEG backend. FFV1 is critical — any lossy codec would alter pixel values during compression and destroy the hidden LSB data.

- Input :

  input.mp4 file path

- Output :

  output.avi encoded video file

8. Business Use Cases

- Example scenarios:

  1. **Covert Corporate Communication** A whistleblower inside an organization needs to send sensitive documents to a journalist without triggering network monitoring systems that flag encrypted traffic. They hide the message inside a regular video file and share it through a normal file sharing platform. The transfer appears as a routine video file transfer, attracting no attention.
  2. **Digital Watermarking** A content creator embeds an invisible ownership identifier inside their video before publishing it online. If the video is stolen and redistributed, the creator can decode the watermark to prove original ownership without any visible mark on the video.
  3. **Secure Field Communication** Intelligence operatives in the field need to communicate without revealing that secure communication is occurring. Encoded messages hidden inside tourist videos posted to social media platforms bypass surveillance systems looking for encrypted traffic.

9. Application Use Cases

- System/user interaction scenarios

Encode a Message

  - Actor: Sender
  - Workflow: Sender runs Encoding.java → enters secret message → program generates key → embeds message into video → output.avi is produced → sender transfers output.avi to receiver via any channel (email, Google Drive, USB)
  - Expected Outcome: Receiver gets output.avi which looks identical to the original video

Decode a Message

  - Actor: Receiver
  - Workflow: Receiver runs Decoding.java with output.avi → program reads frame 0 → recovers key and length → reconstructs frame selection → extracts characters → prints hidden message
  - Expected Outcome: Original secret message is printed exactly as entered by sender

Cross-Machine Transfer

  - Actor: Sender and Receiver on different machines
  - Workflow: Sender encodes on laptop → uploads output.avi to Google Drive → Receiver downloads on a different machine → runs decoder → recovers message
  - Expected Outcome: Message recovered perfectly on a machine that was never involved in encoding

10. Future Scope

- Increased key space — upgrade from 16-bit to 32-bit or 64-bit key to increase brute-force resistance from 65,536 to over 4 billion possible keys
- AES encryption layer — encrypt the message with AES before embedding, combining cryptography and steganography for double security
- Unicode support — extend beyond ASCII to support all languages and emoji using 16-bit character encoding
- Web UI — a browser-based interface to upload videos, enter messages, and download encoded output without using the command line
- Capacity scaling — support longer messages by embedding multiple bits per pixel instead of one
- Steganalysis resistance — implement techniques to make the LSB modifications statistically indistinguishable from natural pixel noise
- Audio steganography — extend the same LSB principle to audio files
- Mobile app — Android/iOS application for on-device steganographic communication
- Automated key exchange — use asymmetric cryptography (RSA) to securely exchange the decoding key separately from the video
- Enhancements

11. Challenges Faced

- Challenge 1

  VideoWriter silent failure The original code used VideoWriter.fourcc('D','I','B',' ') which silently failed on Windows OpenCV 4.12 — it opened the CV_IMAGES backend instead of writing a video, producing an empty output file. No error was thrown, making it extremely difficult to diagnose. Resolved by switching to Videoio.CAP_FFMPEG backend explicitly and implementing a multi-codec fallback loop (FFV1 → HFYU → RGBA).

- Challenge 2

  Missing FFMPEG DLL Even after fixing the codec, all lossless codecs failed because the OpenCV installation was missing opencv_videoio_ffmpeg4120_64.dll. The standard OpenCV installer does not always include this file. Resolved by downloading the full official OpenCV 4.12.0 Windows release and copying the FFMPEG DLL into the project folder.

- Challenge 3

  Key and length region overlap The original storeKey wrote to columns 0 onwards and storeLength wrote to columns 3 onwards — meaning bits 9–15 of the key were overwritten by the length field before decoding. This caused the recovered key to always be wrong, making frame selection completely different on decode. Resolved by moving storeLength to start at column 6, giving each field its own non-overlapping 6-pixel region.

- Challenge 4

  Embed region collision with metadata embedLetter originally started at column 10, which fell inside the length metadata zone (columns 6–11). Characters were being written on top of metadata. Resolved by moving the embed start point to column 12.

- Challenge 5

  PowerShell argument parsing When running Java with -Djava.library.path=. inside PowerShell's Measure-Command, the dot was interpreted as a path separator causing ClassNotFoundException. Resolved by wrapping the argument in single quotes.

12. Outcome

- Secret messages are successfully hidden and recovered with 100% accuracy
- Encoding time: ~9 seconds for a 329-frame 1280×534 video
- Decoding time: ~1 second for the same video
- Pixel modification rate: 0.00044% per character
- Maximum colour deviation: ±1 out of 255
- Zero perceptible visual difference between original and encoded video
- Successfully demonstrated cross-machine decoding via Google Drive transfer
- Output file size comparable to original (lossless FFV1 codec)

13. Conclusion

This project successfully demonstrates that video steganography is a practical and viable method for covert communication. By leveraging the LSB substitution technique across pseudorandomly selected frames, secret messages can be hidden inside ordinary video files with no visible trace. The self-contained key design eliminates the need for any separate key exchange, making the system simple to use while maintaining security through obscurity.

The project provided hands-on experience with OpenCV's Java bindings, lossless video codec selection, bit-level pixel manipulation, and debugging native library integration on Windows. The challenges encountered — particularly the silent VideoWriter failure and the metadata region overlap — developed strong diagnostic and debugging skills.

The system is a functional proof of concept that can be extended with encryption, a web interface, and stronger key management to become a production-grade covert communication tool.

14. References

- OpenCV Documentation — https://docs.opencv.org/4.12.0/
- OpenCV Java Tutorials — https://docs.opencv.org/4.x/d9/d52/tutorial_java_dev_intro.html
- OpenCV 4.12.0 Windows Release — https://github.com/opencv/opencv/releases/tag/4.12.0
- Provos, N. & Honeyman, P. (2003). Hide and Seek: An Introduction to Steganography. IEEE Security & Privacy
- Fridrich, J. (2009). Steganography in Digital Media. Cambridge University Press
- FFV1 Lossless Codec Specification — https://ffmpeg.org/ffv1.html
- Java Random class documentation — https://docs.oracle.com/en/java/docs/api/java.base/java/util/Random.html

- ## 15. Appendix

Key Pixel Layout — Frame 0

```
Column:  0       1       2       3       4       5       6       7       8       9       10  11  12+

         |←————— KEY (16 bits) ———→|←————— LENGTH (16 bits)————————→|

         |          cols 0-5, 3 channels each  |                cols 6-11 , 3 channels each      |

                     = 18 slots, 16 used                            = 18 slots, 16 used       col 12
```

Run Commands

 # Compile

javac -cp ".;opencv-4120.jar" Encoding.java

javac -cp ".;opencv-4120.jar" Decoding.java

 # Encode

java --enable-native-access=ALL-UNNAMED -cp ".;opencv-4120.jar" -Djava.library.path=. Encoding input.mp4 output.avi

 # Decode

java --enable-native-access=ALL-UNNAMED -cp ".;opencv-4120.jar" -Djava.library.path=. Decoding output.avi

 # Encoding

Enter secret message: Hello

Generated key: 20477

Video: 1280x534 @ 23.97fps, 329 frames

Using codec: FFV1

Done. Encoded 329 frames into output.avi

Decoding

Opened output.avi: 329 frames

Recovered key: 20477

Message length: 5

Hidden message: Hello

#UI/IX

A - Encoding

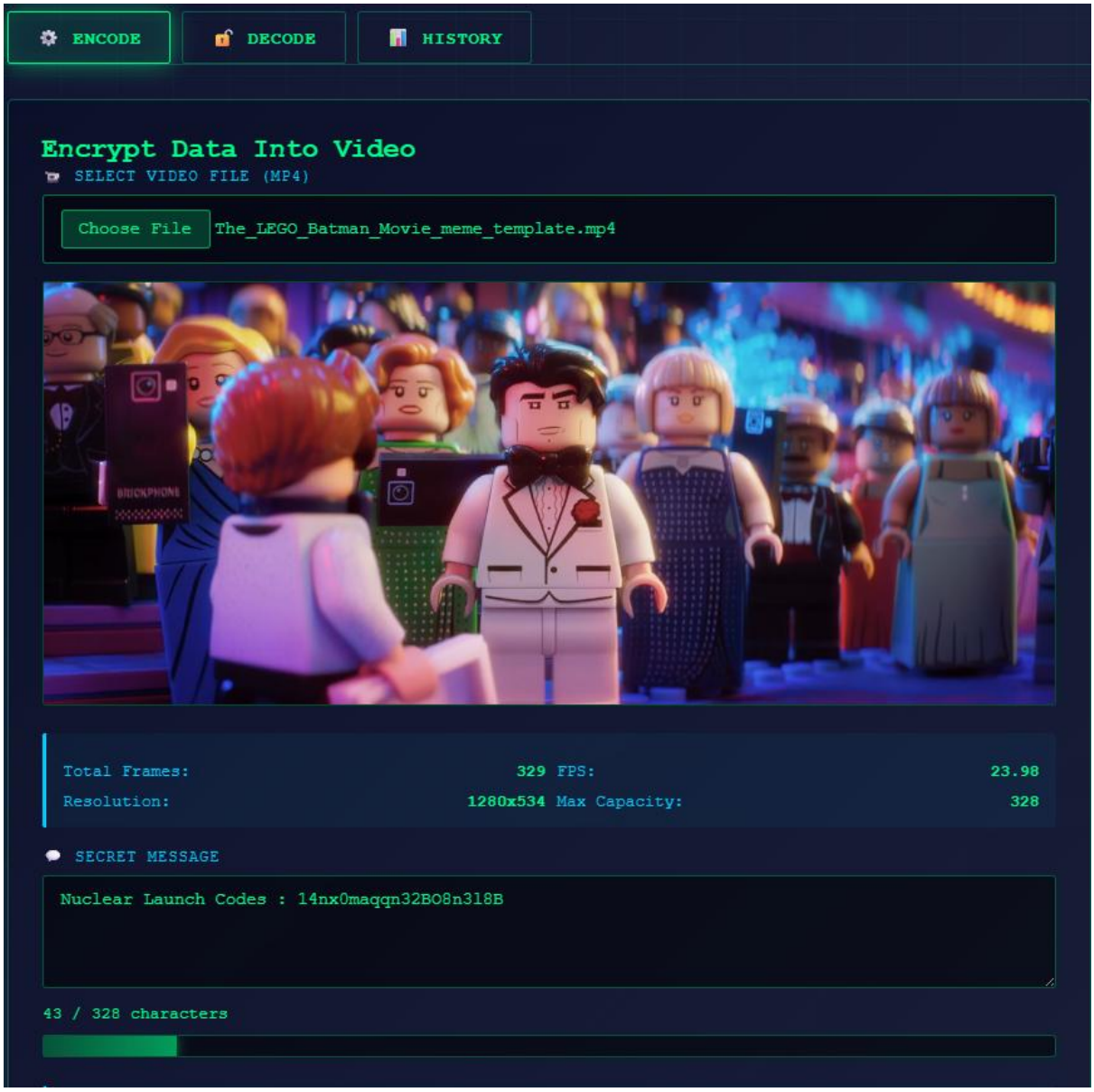

Fig 15a Encoding Demonstration

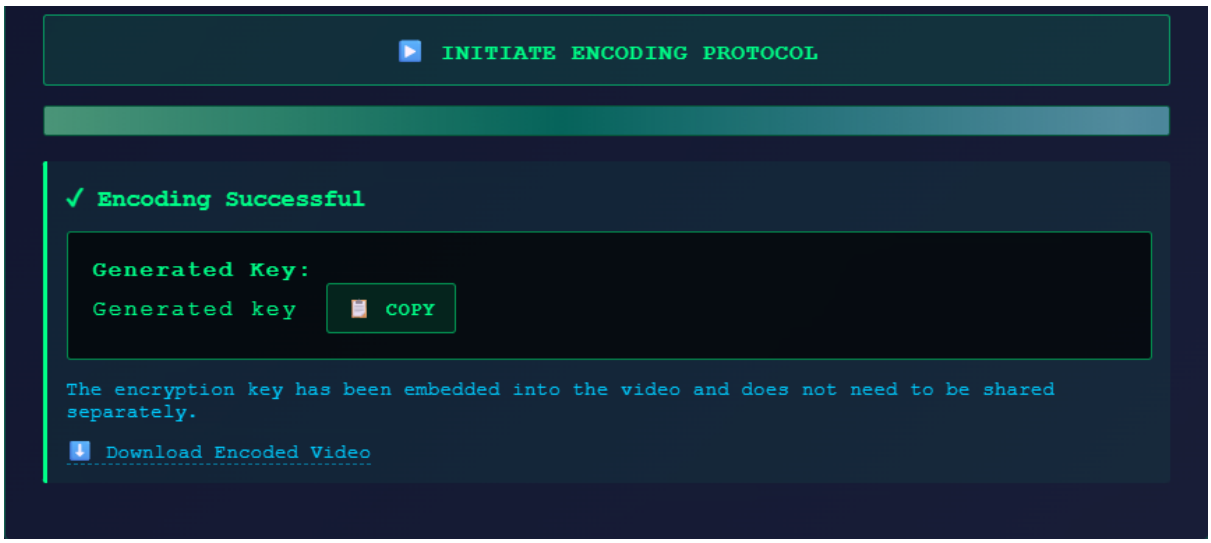

Fig 15b Generating Key

B – Decoding

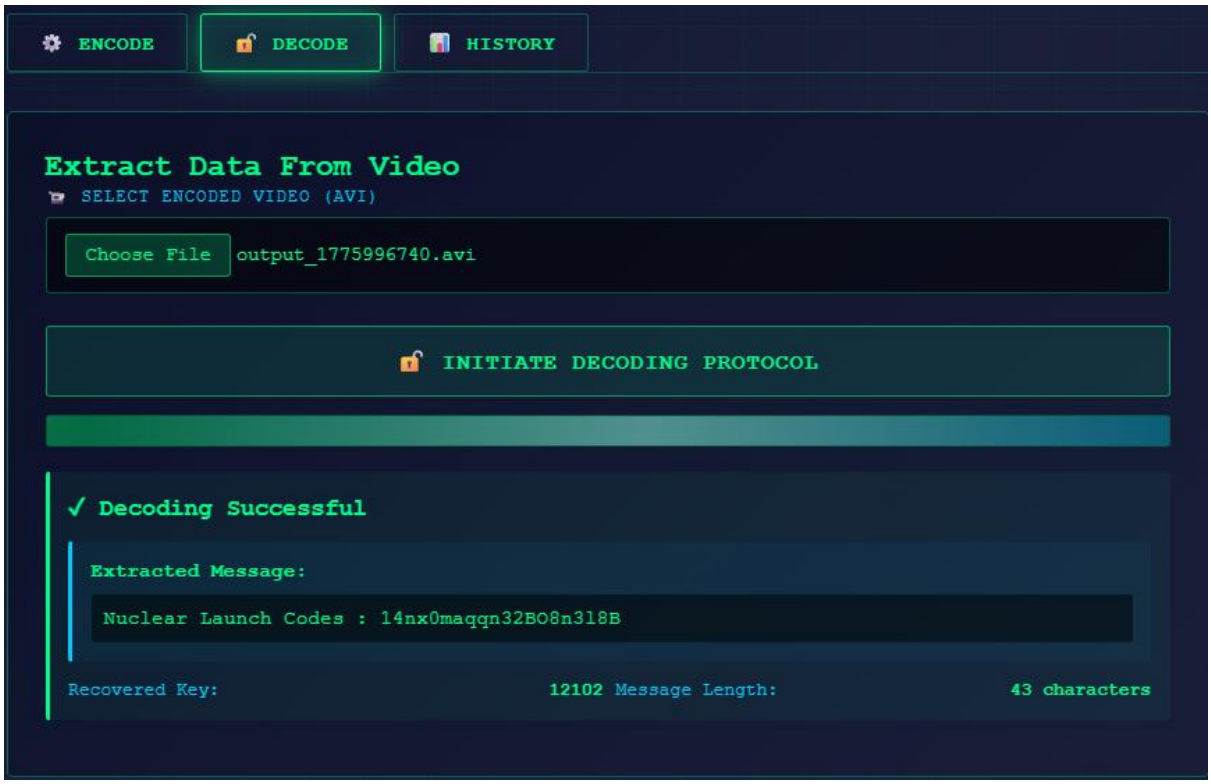

Fig 15c Decoding the text

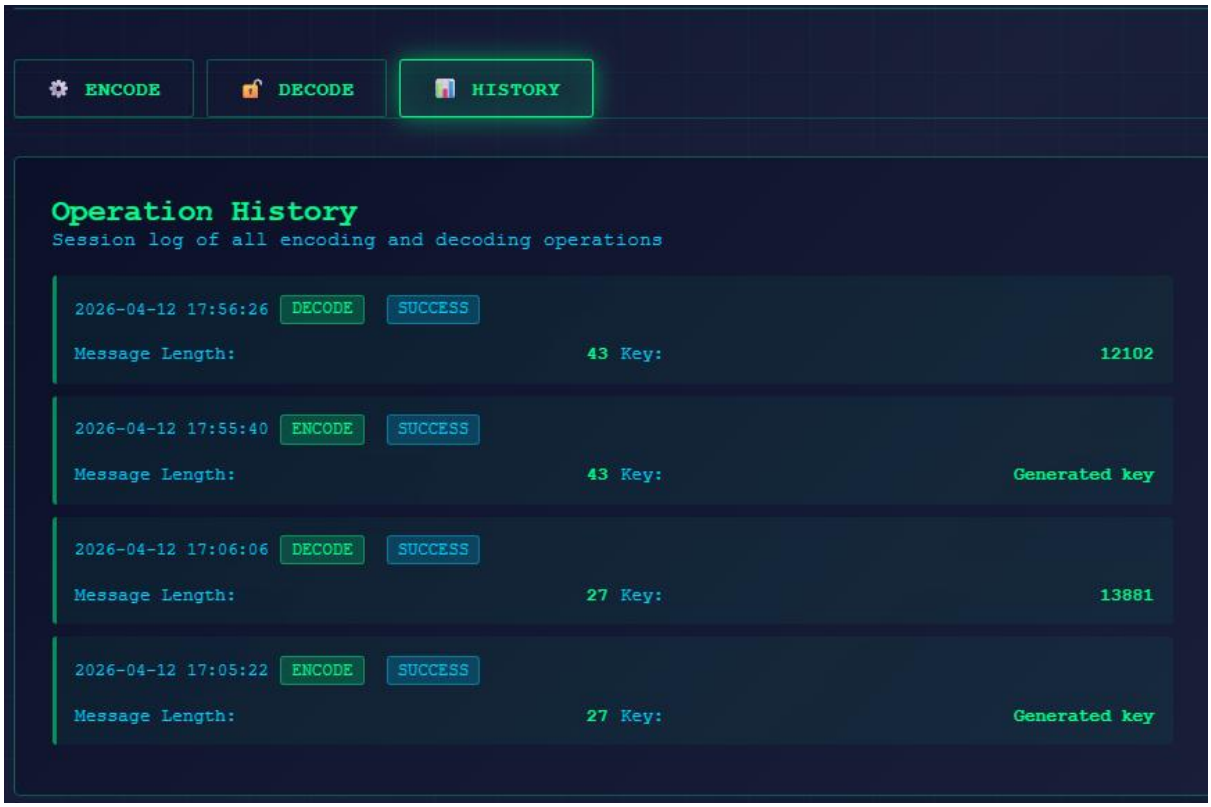

Fig 15d History Logs